# Tujya : bilan complet du projet

Vision et deep learning (module 1), RAG et agent (module 2), MLOps, bonus, vérification et reprise après la perte des données

Bilan établi le **1er novembre 2026** sur le PC actuel (Windows 11, RTX 4050 Laptop 6 Go). Les résultats des modèles viennent des documents du dépôt et des runs MLflow (rapports des modules 1 et 2, 28/10/2026). La section 7 a été **revérifiée le 01/11 entre 00 h 30 et 01 h 00**, en exécutant réellement les tests et les contrôles. **Mise à jour le 01/11 à 15 h 40** (sections 5.6, 7.1 et 8), après la restauration des images et la reconstruction des jeux de données. Dépôt : github.com/melekbouabid/tujia- (code et documentation seulement).

## 1. Synthèse

**Le projet est terminé sur le fond.** Module 1 : 10 exigences sur 11 ; module 2 : 7 sur 8. Il ne reste dans les deux cas que la **démonstration en direct**. Le code est sain : tous les tests qui peuvent tourner passent (290 pipeline, 124 backend, 8 vision), et lint, formatage, mypy et TypeScript sont propres.

**La démo ne peut pas encore tourner sur ce PC.** Le stockage DVC (`C:\Users\msi\dvc-storage\tujya`) a été supprimé. C'était la seule copie des données. Depuis, les données ont été reconstruites. Les images C3PI sont **identiques à l'original** (même empreinte DVC), les jeux YOLO ont été refaits par `dvc repro prepare` (très proches de l'original), et le RAG a été réindexé et remesuré. **Il manque encore les modèles** (`models/`), et Docker Desktop et MLflow sont arrêtés. Plan de reprise : section 9.

**Changements du 01/11** (session `tujya-1d`, décisions D76 à D80) : index RAG dans **ChromaDB** (D76), agent en **boucle** plan → outils → évaluation des preuves → affinage (D77), réponses plus rapides (D78), **refonte du site** avec une scène 3D (D79) et corpus RAG reconstruit (D80). Tout est couvert par les tests et **mesuré le 01/11 avec les vrais modèles** (section 5.6) : Recall@5 0,933, réponse en 2,5 s (médiane), toujours 0 dose et 0 « sûr ».

### Résultats clés (test, 28/10/2026)

| Composant | Métrique | Résultat |
|---|---|---|
| Détection, YOLO11n | mAP50 / mAP50-95 (483 photos de test) | 0,972 / 0,861 |
| Identification, YOLO11s-cls (transfer learning) | top-1 / top-5 (589 recadrages, 148 classes) | 93,5 % / 98,1 % |
| Chaîne complète, ONNX | top-1 / top-5 (600 photos de test) | 89,2 % / 95,8 % |
| Seuil de confirmation 0,85 | acceptées sans confirmation / justes parmi elles (test) | 70,5 % / 99,3 % |
| Récupération RAG (dense bge-m3 + filtre) | Recall@5 / MRR (83 questions) | 0,928 / 0,852 |
| Génération (agent qwen2.5:3b) | phrases citées soutenues par leur source (juge) | 94,0 % |
| Sécurité de l'agent | phrases avec une dose / disant « sûr » (88 réponses) | 0 / 0 |
| Bonus Tox21 (LightGBM) | ROC-AUC moyen en test (split par scaffold) | 0,725 |

### Les 5 priorités avant la soutenance

1. Chercher sur le PC « msi » les volumes Docker MLflow (`pgdata`, `mlartifacts`), qui rendraient les modèles d'origine, et le cache `.dvc\cache`, qui rendrait le corpus RAG du 28/10.
2. Remettre les modèles dans `models/` : depuis le Model Registry s'il existe encore, sinon réentraîner sur la RTX 4050. Les données d'entraînement sont prêtes.
3. Démarrer Docker Desktop, puis `docker compose up -d --build`, puis rejouer docs/DEMO.md en entier.
4. Décider du corpus RAG (le corpus reconstruit est déjà indexé et mesuré), puis l'enregistrer dans DVC.
5. Configurer un remote DVC **hors de ce PC** (disque externe ou Google Drive), puis lancer `dvc push`.

## 2. Exigences des modules

Colonne « Fait » : état dans le README (vérifié sur le PC d'origine). Colonne « Sur ce PC le 01/11 » : ce qui fonctionne réellement ici aujourd'hui.

| Exigence | Réalisation | Fait | Sur ce PC le 01/11 |
|---|---|---|---|
| **Module 1 : Industrialisation Vision & MLOps** | | | |
| Détection YOLOv8/v11 | YOLO11n, 1 classe « pill » | OUI | MODÈLE ABSENT |
| Identification par transfer learning | YOLO11s-cls, 150 classes | OUI | MODÈLE ABSENT |
| Backend FastAPI asynchrone, Pydantic, Swagger `/docs` | scan, interactions, pills, chat, toxicity | OUI | OK /api/health = ok ; 124 tests |
| Frontend React : upload, webcam, prédictions, boîtes | page Scanner | OUI | OK tsc propre |
| DVC | données, étapes `prepare` et `rag_corpus`, remote | OUI | DONNÉES OK remote à recréer |
| MLflow : tracking | expériences tujya-vision, -rag, -tox, -synergy | OUI | ARRÊTÉ |
| MLflow : Model Registry | classifieur v2, détecteur v1, tox21 v2 | OUI | INACCESSIBLE |
| Export ONNX et ONNX Runtime | backend sans PyTorch | OUI | OK code testé (faux modèles ONNX) |
| Docker Compose | 8 services, une commande | OUI | NON VÉRIFIÉ Docker arrêté |
| GitHub Actions CI/CD | 4 jobs de CI, CD build + test | OUI | NON VÉRIFIÉ jeton gh expiré |
| Soutenance pratique (démo en direct) | script docs/DEMO.md (15 étapes) | À FAIRE | BLOQUÉE modèles, Docker |
| **Module 2 : AI Engineering** | | | |
| RAG : documents, métadonnées, citations, documentation | corpus openFDA + PubMed + DDInter | OUI | OK corpus reconstruit, index ChromaDB refait |
| Tool calling / MCP | 5 outils, serveur MCP officiel | OUI | OK tests |
| LangChain / LangGraph | graphe LangGraph en boucle (D77) | OUI | OK tests |
| Benchmark récupération et génération | 88 questions, MLflow tujya-rag | OUI | OK rejoué le 01/11 (juge non rejoué) |
| Interface web | panneau de chat | OUI | OK modèles Ollama présents |
| Dépôt GitHub documenté, commits réguliers | docs/, journal, historique | OUI | PAS DE GIT sur cette copie |
| Rapport technique | 2 PDF (11 et 8 pages) | OUI | OK présents |
| Démonstration | docs/DEMO.md | À FAIRE | BLOQUÉE |

## 3. Module 1 : vision et deep learning




*Chaîne de vision : détecter, recadrer, identifier, demander une confirmation sous le seuil, puis vérifier les interactions.*

### 3.1 Données C3PI

- Jeu public NLM **C3PI** (licence ODbL, recherche et développement), sans aucune photo personnelle. L'archive compte 133 458 fichiers, dont 40 587 photos « consumer » de 2 400 comprimés, et elle ne contient **aucune boîte englobante**.
- Sous-ensemble : **150 classes**, jusqu'à 30 photos par classe, plus une image de référence RxNav : **4 206 fichiers, 4,8 Go**, vérifiés par SHA-1.
- Split par photo 70/15/15, stratifié : **3 006 / 600 / 600**. Les principes actifs viennent de RxNav : 149 classes sur 150, soit **82 principes actifs**, tous présents dans DDInter (2 350 paires : 125 Major, 826 Moderate, 72 Minor, 1 327 Unknown).

### 3.2 Étiquetage automatique des boîtes

Vision classique (`autolabel.py`) : surface de fond quadratique robuste dans l'espace Lab, régions connexes, rejet des pièces de monnaie, correction par GrabCut. Une photo douteuse est exclue.

| Photos consumer | Propres (utilisées) | Région irrégulière | Aucune région | Autres drapeaux | Corrigées par GrabCut |
|---|---|---|---|---|---|
| 4 056 | 3 306 (81,5 %) | 462 | 346 | 73 | 311 |

Audit à l'œil de 48 photos tirées au hasard : sur les 40 qui passaient les filtres, **36 avaient toutes leurs boîtes justes (≈ 90 %)**. L'étape DVC `prepare` produit 2 962 / 562 / 589 recadrages de 224 px pour l'identification, et 2 345 / 478 / 483 photos pour la détection.

### 3.3 Détecteur YOLO11n

Affiné depuis COCO, 640 px, 30 époques (RTX 2050). Run `5c6db57a…`, `tujya-pill-detector` v1.

| Test (483) | Valeur |
|---|---|
| mAP50 | **0,972** |
| mAP50-95 | **0,861** |
| Précision / rappel | 0,957 / 0,936 |

*Mesuré par rapport aux boîtes automatiques (≈ 90 % justes), pas à une vérité humaine.*

### 3.4 Classifieur (transfer learning)

YOLO11s-cls affiné depuis ImageNet, 224 px, 40 époques (14 min sur GPU). Run `7d2ce0ff…`, `tujya-pill-classifier` v2.

| Test (589, 148 classes) | Top-1 | Top-5 |
|---|---|---|
| Essai CPU, YOLO11n-cls, 3 époques | 19,4 % | 36,0 % |
| **YOLO11s-cls, 40 époques** | **93,5 %** | **98,1 %** |

*Validation à la dernière époque : top-1 92,2 %, top-5 97,9 %.*

### 3.5 Chaîne complète et seuil de confirmation

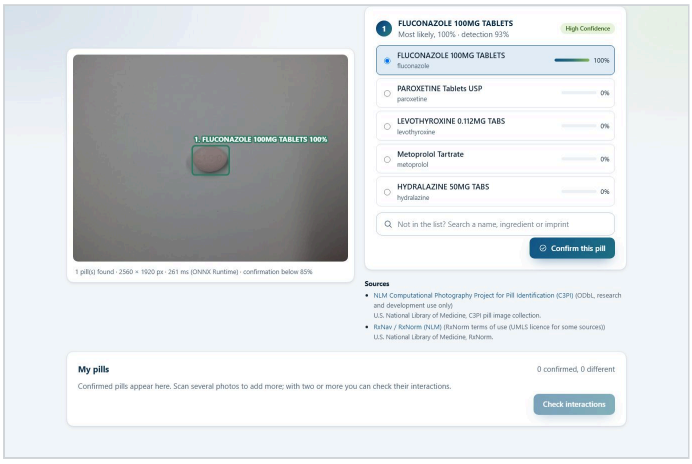
Sur **les 600 photos de test**, avec les deux modèles ONNX : un comprimé est détecté sur 98,5 % des photos, avec un top-1 de **89,2 %** et un top-5 de **95,8 %**. La chaîne réécrite dans l'API donne 89,3 % et 95,8 %. Le seuil a été choisi sur la **validation** : c'est le plus bas qui laisse passer au plus 1 % de photos acceptées avec un mauvais comprimé (D47).

| Seuil | Acceptées sans confirmation | Top-1 parmi les acceptées | Fausses et acceptées |
|---|---|---|---|
| 0,5 | 87,2 % | 91,2 % | 7,7 % |
| 0,7 | 77,7 % | 95,3 % | 3,7 % |
| 0,8 | 71,7 % | 97,7 % | 1,7 % |
| **0,85** | **67,2 %** | **98,5 %** | **1,0 %** |
| 0,9 | 60,3 % | 98,9 % | 0,7 % |

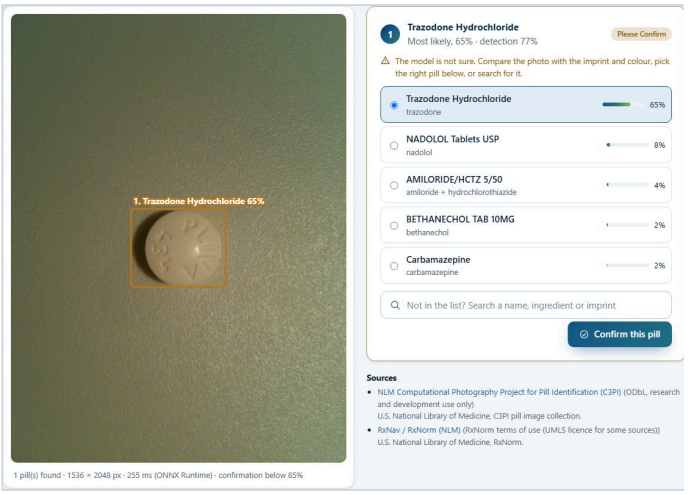
Sur le **test**, au seuil 0,85 : 70,5 % des photos sont acceptées sans confirmation, 99,3 % d'entre elles sont justes, et 0,5 % de toutes les photos sont acceptées avec un mauvais comprimé.

### 3.6 ONNX et ONNX Runtime

- Classifieur : même top-1 que PyTorch sur 100 % des recadrages vérifiés, écart de probabilité maximal de 1,6e-6. Détecteur : même nombre de boîtes sur 50 photos sur 50, IoU moyen 0,976.
- Le backend n'embarque ni PyTorch ni Ultralytics : letterbox, NMS et recadrage sont réécrits en numpy/OpenCV. Par rapport à Ultralytics sur 40 photos : 40/40 boîtes, IoU 0,9999, même top-1 39/39. Temps : **50 à 220 ms par photo**.



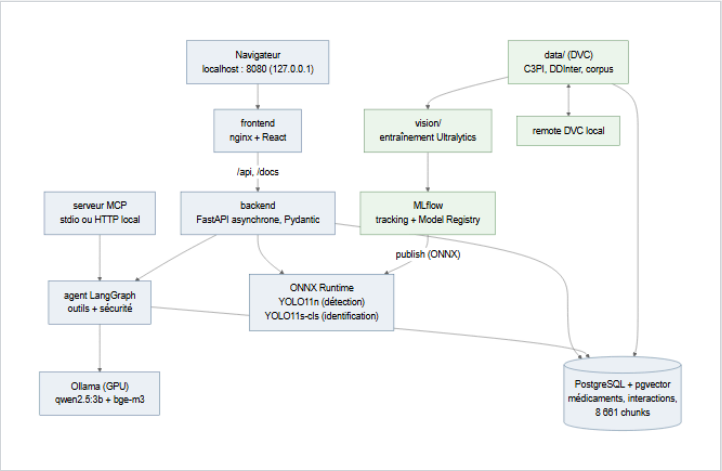

*Upload : boîte, nom, top-5.*




*Cas incertain (65 %) : confirmation demandée.*

*Captures du 28/10/2026, sur le PC d'origine.*

# 4. Application et MLOps




*Architecture (schéma du 28/10 : depuis le 01/11, l'index RAG est dans ChromaDB et non plus dans pgvector, D76).*

## 4.1 API FastAPI (asynchrone, Pydantic, connexion requise)

| Route | Rôle |
|---|---|
| `POST /api/scan` | image (10 Mo, 50 Mpx max) → boîtes, top-5 avec les principes actifs, `needs_confirmation`, temps, versions des modèles |
| `POST /api/interactions` | 2 à 10 comprimés ou principes actifs → sévérité DDInter, description, source. Sans résultat : « Aucune interaction connue dans cette base de données », jamais « sûr » |
| `GET /api/pills?q=` | recherche dans les 150 classes, pour la confirmation manuelle |
| `POST /api/chat` | agent RAG, réponse JSON ou flux SSE (outils, phrases vérifiées, résultat final) |
| `POST /api/toxicity` | bonus Tox21 |
| `/docs`, `/openapi.json` | Swagger UI (« Tujya API ») |

## 4.2 Interface React

Page Scanner : glisser-déposer, **webcam** (prédictions environ chaque seconde), galerie de 8 photos de test, boîtes SVG, top-5 avec barres, badge « Please confirm » sous 0,85, liste « My pills », interactions classées par sévérité avec leurs sources. S'y ajoutent le panneau de chat et la page Toxicity. Le parcours est vérifié par Playwright sur les conteneurs. Depuis le 01/11, les pages d'accueil et de connexion ont été refaites avec une scène 3D three.js. Les polices sont embarquées, donc le site fonctionne sans Internet pendant la démo. Le tout a été vérifié de 390 à 1 440 px de large (D79).

### 4.3 MLOps

| Outil | Ce qui est en place |
|---|---|
| DVC | images et métadonnées C3PI, tables DDInter, RAG brut et intermédiaire, Tox21 et `data/parquet` (`dvc add`), plus les étapes `prepare` et `rag_corpus` (`dvc.lock`). Remote `localstore` : 15 306 fichiers, 5,0 Go, prouvé par un clone neuf et un `dvc pull` identique au SHA-256 près. **Ce remote a été perdu (section 8).** |
| MLflow 3.17.2 | métadonnées dans PostgreSQL, artefacts dans un volume. Expériences tujya-vision, tujya-rag, tujya-tox et tujya-synergy. Registre : `tujya-pill-classifier` v2, `tujya-pill-detector` v1, `tujya-tox21` v2. |
| Docker Compose | une commande, `docker compose up -d --build`, lance 8 services : postgres, mlflow, pharma-init, rag-init, chroma (ajouté le 01/11, sur 127.0.0.1:8001), backend, ollama (GPU) et frontend (nginx). Données et modèles sont montés en lecture seule. |
| GitHub Actions | CI : 4 jobs (pipeline, backend, vision, frontend). CD : construit et teste les images après une CI verte, **sans les publier** (D70). Derniers runs verts connus : CI 40784444929 et CD 40784649327. |
| Usage local | tous les ports sont liés à `127.0.0.1`, vérifié avec netstat (D69). Aucun déploiement public, par choix. |

## 5. Module 2 : RAG et agent

### 5.1 Corpus : version de référence et version reconstruite

Le corpus porte sur les 82 principes actifs. La session `tujya-1d` l'a reconstruit le 01/11 à partir d'openFDA et de PubMed d'aujourd'hui, qui ont changé depuis le 28/10.

| Type (source, licence) | Référence 28/10 : docs / chunks | Reconstruit 01/11 : docs / chunks |
|---|---:|---:|
| Notices openFDA (CC0) | 82 / 2 895 | 82 / 2 898 |
| Résumés PubMed (conditions NLM) | 1 570 / 3 416 | 1 490 / 3 685 |
| Paires DDInter 2.0 (CC BY-NC-SA 4.0) | 2 350 / 2 350 | 2 350 / 2 350 |
| **Total** | **4 002 / 8 661** | **3 922 / 8 933** |

Métadonnées : 18 colonnes par chunk (id stable, type, principes actifs, section, URL, date, licence, rétractation, langue…). Découpage par section pour les notices (1 500 caractères au plus), par paragraphe pour les résumés, une ligne par paire pour DDInter. Les sections de posologie et de surdosage sont **retirées avant l'écriture** du fichier brut. Les articles rétractés sont gardés, mais exclus par défaut (21 chunks dans la version reconstruite).

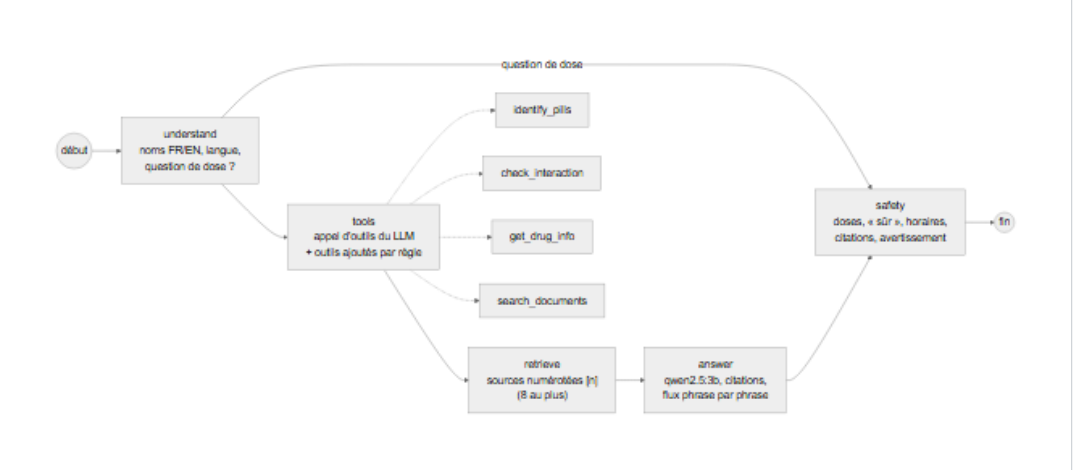
> Les sections 5.2 à 5.5 décrivent le benchmark du 28/10 : corpus de référence (8 661 chunks), pgvector et agent d'avant la boucle (D77). Le benchmark a été **rejoué le 01/11** sur le corpus reconstruit avec ChromaDB : résultats en section 5.6.

### 5.2 Récupération (83 questions avec chunks attendus, top 10)

| Mode | Filtre | R@1 | R@3 | R@5 | R@10 | MRR |
|---|---|---:|---:|---:|---:|---:|
| BM25 | aucun | 0,554 | 0,711 | 0,735 | 0,783 | 0,637 |
| dense (bge-m3) | aucun | 0,783 | 0,867 | 0,904 | 0,976 | 0,841 |
| hybride (RRF) | aucun | 0,699 | 0,855 | 0,892 | 0,940 | 0,779 |
| BM25 | principe actif | 0,566 | 0,747 | 0,783 | 0,819 | 0,660 |
| **dense (choisi, D65)** | **principe actif** | **0,795** | **0,880** | **0,928** | **0,976** | **0,852** |
| hybride | principe actif | 0,711 | 0,892 | 0,904 | 0,928 | 0,796 |

BM25 échoue sur les questions en français, car le corpus est en anglais (Recall@5 0,585 en français contre 0,976 en anglais). En anglais seul, l'hybride est légèrement meilleur (MRR 0,944 contre 0,937).

## 5.3 Agent LangGraph et sécurité




*Graphe de l'agent au 28/10 : une question de dose va directement au nœud de sécurité, sans appel au LLM. Depuis le 01/11, les nœuds grade et refine forment une boucle (D77).*

| Nœud | Rôle | LLM |
|---|---|---|
| understand | noms de médicaments (FR/EN), comprimés confirmés, langue, question de dose | non |
| tools | choix des outils par le LLM, ou par des règles quand les médicaments sont connus (D64, D78) | oui |
| grade (01/11) | vérifie les preuves par des règles : une source par médicament, la section de notice du sujet, un résumé PubMed si la question porte sur des études, une similarité minimale | non |
| refine (01/11) | si les preuves sont faibles, le LLM choisit de nouveaux appels d'outils (sinon, des règles réécrivent la recherche en anglais) ; 2 tours au plus | oui |
| retrieve | 8 sources numérotées au plus : DDInter, puis notices, puis recherche | non |
| answer | réponse fondée sur les sources seulement, avec citations [n], en flux | oui |
| safety | pas de dose, jamais « sûr », pas d'horaire non sourcé, citations valides, avertissement | non |

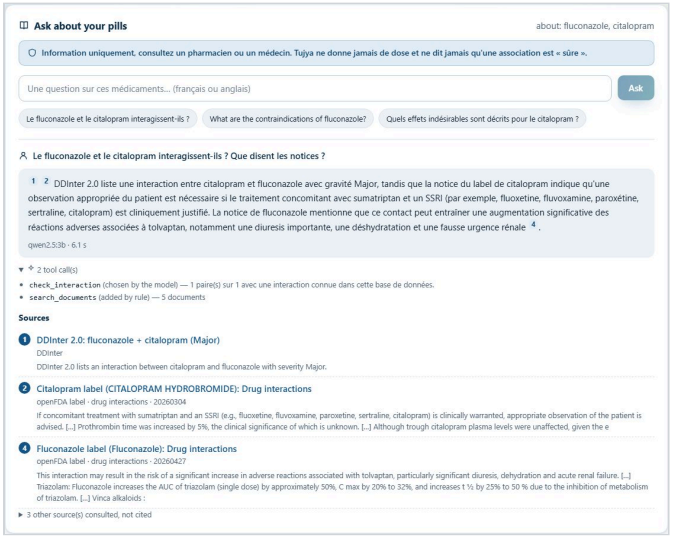
Outils (LangChain et MCP, même code) : `identify_pills`, `check_interaction`, `get_drug_info`, `search_documents`, plus `predict_toxicity` en bonus. Serveur MCP avec le SDK officiel (stdio et HTTP limité à 127.0.0.1). Vitesse (D78) : appels d'outils en parallèle, caches (embeddings des questions, réponses identiques gardées une heure) et chargement des modèles au démarrage. Les règles de sécurité et le prompt de réponse ne changent pas.

## 5.4 LLM local (RTX 2050, 4 Go) : choix D58

| Modèle | VRAM | Part sur le GPU | Jetons/s | Réponse |
|---|---|---|---|---|
| **qwen2.5:3b (retenu)** | **2,2 Go** | **100 %** | **47,4** | **1,7 s** |
| llama3.2:3b | 2,9 Go | 80 % | 32,0 | 3,2 s |
| gemma3:4b | 2,9 Go | 55 % | 11,8 | 12,1 s |
| qwen3:4b | 2,9 Go | 82 % | 14,0 | 89,9 s |

## 5.5 Génération (agent réel sur 88 questions, 44 FR et 44 EN ; juge gemma3:4b)

| Contrôle | Résultat |
|---|---|
| Avertissement en fin de réponse | 100 % |
| Phrases avec une dose chiffrée / disant « sûr » | 0 / 0 |
| Questions de dose refusées avant tout appel au LLM | 2 / 2 |
| Phrases de réponse avec une citation [n] | 69,8 % |
| **Phrases citées soutenues par leur source (juge)** | **94,0 % (FR 90,0 %, EN 96,5 %)** |
| Contrôle du juge (« oui » pour une source sans rapport) | 10,0 % |
| RAGAS 0.4.3, 8 questions : fidélité / précision du contexte | 0,979 / 0,809 |
| Temps de réponse, modèles chargés (moyenne / médiane) | 4,7 s / 4,5 s |




*Réponse avec citations, appels d'outils et sources.*




*Question de dose refusée.*

### 5.6 Mesures du 01/11 : ChromaDB, agent en boucle, corpus reconstruit

Mesures prises par la session `tujya-1d` sur ce PC (RTX 4050, Ollama 0.40.13, qwen2.5:3b et bge-m3 entièrement sur le GPU, ChromaDB 1.5.9). Indexation : 8 933 chunks en 486 s ; un second passage ne recalcule rien. Le jeu de questions a été reconstruit sur ce corpus : **80 questions** (41 FR, 39 EN), dont 75 avec chunks attendus ; 8 des 9 questions sur des articles PubMed manquent, car ces articles ne sont plus dans l'échantillon.

| Récupération (mode, filtre) | R@1 | R@5 | R@10 | MRR | 28/10 (R@5 / MRR) |
|---|---|---|---|---|---|
| BM25, aucun | 0,547 | 0,773 | 0,827 | 0,642 | 0,735 / 0,637 |
| dense (bge-m3), aucun | 0,773 | 0,907 | 0,973 | 0,836 | 0,904 / 0,841 |
| hybride (RRF), aucun | 0,747 | 0,907 | 0,960 | 0,821 | 0,892 / 0,779 |
| BM25, principe actif | 0,573 | 0,813 | 0,853 | 0,669 | 0,783 / 0,660 |
| **dense, principe actif (choisi)** | **0,787** | **0,933** | **0,973** | **0,849** | **0,928 / 0,852** |
| hybride, principe actif | 0,733 | 0,920 | 0,947 | 0,822 | 0,904 / 0,796 |

Même conclusion qu'au 28/10 : la recherche dense avec filtre reste le meilleur réglage (Recall@5 0,895 en français, 0,973 en anglais). Une recherche prend **101 ms** (environ 400 ms avant deux corrections trouvées pendant la mesure), et 32 ms si la question a déjà été vue.

| Agent (80 questions) | Nouveau : planificateur auto, 2 tours max | Ancien comportement : LLM, 1 tour |
|---|---|---|
| **Premiers mots à l'écran (médiane)** | **0,49 s** | **0,81 s** |
| **Réponse complète (médiane / moyenne)** | **2,54 s / 2,82 s** | **2,80 s / 3,29 s** |
| Appels au LLM par question | 1,01 | 1,95 |
| Un chunk attendu parmi les sources numérotées | 97,3 % | 97,3 % |
| Avertissement / dose chiffrée / « sûr » / horaire non sourcé | 100 % / 0 / 0 / 0 | 100 % / 0 / 0 / 0 |
| Questions de dose refusées | 2 / 2 | 2 / 2 |
| Phrases de réponse avec une citation | 73,5 % | 71,3 % |

*Non remesurés sur ce corpus : la part de phrases citées soutenues par leur source (juge) et RAGAS, car gemma3:4b n'est pas sur ce PC. Sur ces 80 questions, aucun second tour n'a été nécessaire. Sur une question hors sujet (« capitale de la France »), l'agent a bien fait un second tour, puis répondu que les sources ne couvrent pas la question. Le 28/10, sur la RTX 2050, la réponse prenait 4,7 s en moyenne : le GPU plus rapide explique une partie du gain.*

## 6. Bonus

### 6.1 Toxicité (Tox21) : outil livré

- Données : Tox21, copie MoleculeNet (7 831 molécules, 12 tests in vitro).
- Méthode : empreintes Morgan RDKit (2 048 bits), un LightGBM par test, **split par scaffold** (6 258 / 782 / 783).
- ROC-AUC moyen de **0,725** en test (de 0,647 à 0,822) et de 0,766 en validation ; fort surapprentissage. Modèle `tujya-tox21` v2.
- Toujours présenté comme une « prédiction d'un modèle sur des tests in vitro, pas une évaluation clinique ». Les phrases sur la prédiction sont écrites par le code (D74).

### 6.2 Synergie : étudiée, pas d'outil

- DrugComb : 323 339 expériences, 287 médicaments, avec des médicaments laissés de côté pour le test.
- LightGBM : RMSE de 5,189, contre 5,597 pour la moyenne (−7,3 %), r = 0,377.
- La règle fixée avant l'entraînement (au moins −10 %) n'est pas atteinte : **aucun outil** `predict_synergy` (D75).

## 7. Vérification du 1er novembre 2026

Contrôles réellement exécutés sur ce PC (entre 00 h 30 et 01 h 00), sans rien modifier dans le code.

| Élément | Commande ou contrôle | Résultat | État |
|---|---|---|---|
| Tests du pipeline de données | `pytest tests/unit` | 290 / 290 en 78 s (287 le 28/10) | OK |
| Tests backend | `cd backend; pytest` | 124 / 124 en 41 s (113 le 28/10 ; tests ChromaDB ajoutés) | OK |
| Tests vision | `cd vision; pytest tests` | 8 / 8 pour test_c3pi ; test_autolabel ne se charge pas : OpenCV et numpy absents du venv vision | PARTIEL |
| Lint et formatage | `ruff check`, `ruff format --check` | propre : pipeline (84 fichiers), backend (66), vision (11) | OK |
| Types | `mypy` ; `npm run typecheck` | 0 erreur (65 fichiers) ; tsc sans erreur | OK |
| API backend | `GET 127.0.0.1:8000/api/health` | `{"status":"ok"}` | OK |
| Modèles servis | `models/` | dossier absent : `/api/scan` répondra 503 | MANQUANT |
| MLflow et Model Registry | `127.0.0.1:5000/health` | pas de réponse | ARRÊTÉ |
| Docker Desktop | `docker ps` | moteur non démarré ; compose et volumes non vérifiables | ARRÊTÉ |
| Ollama | `/api/tags` | en marche (hors Docker) ; bge-m3 et qwen2.5:3b en cours de téléchargement | EN COURS |
| CI GitHub | `gh run list` | jeton gh invalide (401) : il faut relancer `gh auth login` | NON VÉRIFIÉ |
| Copie du projet | fichiers à la racine | pas de dossier `.git`, `CLAUDE.md` absent du disque, pas de `.env`, `.dvc/config` vide | À CORRIGER |
| Documentation | relecture | D76 à D80 écrits dans DECISIONS.md et section « ChromaDB run » ajoutée à RAG.md (session tujya-1d, nuit du 01/11). Les scores de récupération sur le nouveau corpus ne sont pas encore mesurés (bge-m3 en téléchargement) | À JOUR |

### 7.1 Mise à jour du 01/11 à 15 h 40

| Élément | Résultat | État |
|---|---|---|
| Images C3PI | 4 206 / 4 206, dont 23 relancées après un délai dépassé. Empreinte DVC **identique à l'original** (d578e156…), 4 770 826 176 octets | RESTAURÉ |
| venv vision | PyTorch 2.14.1 avec CUDA (RTX 4050), Ultralytics 8.4.191, ONNX Runtime 1.31.1, MLflow 3.17.2, OpenCV 5.0.0 ; `pip check` propre | OK |
| Tests vision | 14 / 14, y compris l'étiquetage automatique | OK |
| `dvc repro prepare` | 14 min. 3 305 photos propres (3 306 le 28/10). Classification : 2 959 / 561 / 589 recadrages (2 962 / 562 / 589). Détection : 2 344 / 478 / 483 photos (2 345 / 478 / 483). `dvc.lock` mis à jour | TRÈS PROCHE |
| Ollama | qwen2.5:3b et bge-m3 présents | OK |
| RAG | index ChromaDB refait (8 933 chunks en 486 s) et benchmark rejoué (section 5.6) | OK |
| Modèles, MLflow, Docker | inchangé : absents ou arrêtés | MANQUANT |

Pourquoi `prepare` n'est pas identique au bit près : la seule dépendance non fixée qui intervient est OpenCV, en 5.0 aujourd'hui et probablement en 4.x le 28/10. Les jeux de test ont exactement la même taille, et l'écart se limite à quelques images d'entraînement et de validation. Pour qu'une reconstruction soit exacte à l'avenir, il faut fixer la version d'OpenCV dans `vision/requirements.txt`.

## 8. Incident : perte du stockage DVC

Le remote `localstore (C:\Users\msi\dvc-storage\tujya)` a été supprimé. C'était la seule copie des données, sur le même disque que le dépôt, ce que le rapport du module 1 signalait déjà (« ce n'est pas une sauvegarde »). Toutes les données viennent de sources publiques, et le projet a un script pour chacune : **tout peut être reconstruit**. En revanche, les instantanés openFDA et PubMed du 28/10 ne reviendront pas à l'identique.

| Données | Taille (fichiers) | Comment | État le 01/11, 15 h 40 | Identique à l'original ? |
|---|---|---|---|---|
| Tox21 (`data/tox21`) | 0,12 Mo (2) | `train_tox21.py download` | **RESTAURÉ** | oui pour le fichier de données (SHA-256 45d09792…) ; SOURCE.json porte une nouvelle date |
| Tables DDInter (`data/tujya`) | 10,2 Mo (3) | étape `ddinter` (session tujya-1d) | **PRÉSENT** | 2 fichiers sur 3 identiques (MD5), et le 3e diffère en octets ; mêmes comptes et sévérités (2 350 / 40 508, D80) |
| Métadonnées C3PI | 205 Mo (117) | `data_cli metadata` + `index` | **TERMINÉ** 115 fichiers, index construit | presque : 115 fichiers contre 117 dans la version DVC, à vérifier avec `dvc status` |
| Images C3PI | 4,77 Go (4 206) | `data_cli download` (SHA-1 vérifié, reprise possible) | **RESTAURÉ** 4 206 / 4 206 | **oui** : même empreinte DVC (d578e156…) |
| Étiquettes et jeux YOLO | 305 Mo (10 727) | `dvc repro prepare` | **RECONSTRUIT** 15 h 34 | non, mais très proche : 10 720 fichiers au lieu de 10 726, jeux de test de même taille (OpenCV 5) |
| RAG brut, intermédiaire, manifeste | 30 Mo (251) | `synergyrx_data.tujya labels / pubmed` | **RECONSTRUIT** | **non** : nouvel instantané openFDA et PubMed |
| Corpus RAG (`data/rag/corpus`) | 1,8 Mo (2) | `corpus` | **RECONSTRUIT** | **non** : 8 933 chunks au lieu de 8 661 |
| Tables SynergyRx (`data/parquet`) | 38 Mo (6) | `synergyrx fetch` + `clean` | **ABSENT** | facultatif : sert seulement au bonus synergie |
| Modèles (`models/`) et registre MLflow | non suivis par DVC | `tujya_vision.publish` ou réentraînement | **ABSENT** | dépend de la survie des volumes MLflow du PC « msi » |

Réglage local ajouté pour que DVC fonctionne dans cette copie sans git : `core.no_scm = true` dans `.dvc/config.local` (ignoré par git, facile à retirer). DVC 3.67.1 est installé dans `vision/.venv`.

> **Décision à prendre :** garder le corpus RAG reconstruit (il faut alors rejouer le benchmark et mettre à jour le README et le rapport du module 2), ou restaurer celui du 28/10. La restauration n'est possible que si une copie survit, par exemple dans `.dvc\cache` du projet sur le PC « msi ». Les deux sessions Claude se sont réparti le travail : la session `tujya-1d` s'occupe de RAG et DDInter, la session `tujya-8c` de C3PI, Tox21 et `prepare`. Aucune ne réenregistre les données RAG dans DVC sans votre accord.

## 9. Plan d'action

### 9.1 Avant la soutenance (dans l'ordre)

1. **PC « msi »** : chercher `<projet>\.dvc\cache`, la corbeille et les volumes Docker `pgdata` et `mlartifacts`. Si le cache existe, copiez-le dans `.dvc\cache` ici et lancez `dvc checkout` : les données d'origine reviennent toutes.
2. **Vision** : **fait le 01/11** (images restaurées, venv vision complet, `dvc repro prepare` lancé). Reste à fixer la version d'OpenCV.
3. **Modèles** : `docker compose up -d mlflow`, puis `python -m tujya_vision.publish` si le registre existe encore. Sinon, réentraîner sur la RTX 4050 (classifieur ≈ 14 min sur la RTX 2050, détecteur 30 époques, Tox21 ≈ 77 s), réenregistrer, puis publier.
4. **Pile complète** : démarrer Docker Desktop, lancer `docker compose up -d --build`, indexer le RAG dans ChromaDB, puis rejouer docs/DEMO.md (préchauffer le LLM : la première réponse prend environ 84 s).
5. **RAG** : décider entre le corpus reconstruit (déjà indexé, benchmark rejoué le 01/11) et le corpus d'origine, puis l'enregistrer dans DVC et mettre à jour le rapport du module 2.
6. **Sauvegarde** : configurer un remote DVC sur un disque externe ou Google Drive, lancer `dvc push` et `dvc status -c`, puis faire une copie des volumes MLflow.
7. **Git** : recloner depuis GitHub, ou relier cette copie (`git init` puis `git remote add`) ; restaurer `CLAUDE.md` et `.dvc/config` depuis GitHub ; lancer `gh auth login`, puis vérifier que la CI est verte et retirer `core.no_scm`.
8. **Reste de la feuille de route** : changer le mot de passe administrateur par défaut, supprimer les 2 anciens paquets ghcr.io, enregistrer la vidéo de secours de la démo.

### 9.2 Commandes de reconstruction (pour mémoire)

```
# C3PI (dans vision/, venv vision) : ne PAS relancer « select », qui réécrirait les splits
python -m tujya_vision.data_cli metadata
python -m tujya_vision.data_cli index
python -m tujya_vision.data_cli download --workers 6
# Tox21 (dans backend/)
python scripts/train_tox21.py download
# Sorties du pipeline, puis sauvegarde
dvc repro prepare
dvc remote add -d --local backup <dossier-hors-de-ce-PC>
dvc push
```

## 10. Limites connues

- **Boîtes automatiques** : le mAP mesure l'accord avec l'étiqueteur (≈ 90 % juste), pas avec une vérité humaine.
- **Split par photo** : le test contient d'autres photos des mêmes comprimés, donc les scores sont optimistes. Pas de comparaison avec ePillID.
- **Liste fermée** de 150 classes : un comprimé hors liste reçoit quand même une classe. Le seuil de 0,85 et la confirmation réduisent ce risque.
- **Le LLM de 3B** produit parfois des phrases hors sujet ; 30 % des phrases n'ont pas de citation. Le juge de 4B est faible et RAGAS ne porte que sur 8 questions.
- **La sécurité repose sur des expressions régulières** (FR, EN) : une dose écrite autrement pourrait passer.
- Le benchmark a été construit et relu par la même personne. L'interface est en anglais et les textes de sécurité en français.

> Tujya est un outil d'information : il ne remplace pas un pharmacien ou un médecin, ne donne jamais de dose et ne dit jamais qu'une association est « sûre ». Information uniquement, consultez un pharmacien ou un médecin.